# RELATORIO TECNICO

## Sistema de cadastro de locais urbanos

Disciplina: Estrutura de Dados - ADS, 3o semestre
Linguagem: C11 | Tema B: Locais urbanos
Estruturas: BST e fila dinamica | Estudante: Diego Gallo
Data: 8 de novembro de 2026

## 1. Objetivo e modelagem

O programa cadastra locais urbanos em uma BST ordenada pelo ID. Cada registro guarda ID inteiro, nome, categoria e bairro. Campos textuais usam char[80], permitindo ate 79 caracteres mais o terminador nulo. A entrada remove espacos externos e rejeita textos vazios.

Um no da BST contem o registro e ponteiros para filhos esquerdo e direito. IDs menores ficam a esquerda, maiores a direita; IDs duplicados sao rejeitados. A estrutura linear escolhida e uma fila encadeada independente. Cada no guarda ID positivo e ponteiro para o proximo. A fila permite IDs repetidos e nao exige cadastro correspondente na BST.

O menu oferece cadastro, busca, remocao e percurso em ordem da BST; insercao, remocao e exibicao da fila; e encerramento com liberacao dos nos.

## 2. Organizacao

Os cabecalhos estao em include/ e as implementacoes em src/. bst.c concentra as operacoes da arvore; fila.c implementa a fila; entrada.c valida dados; main.c integra o menu.

## 3. Implementacao

A BST implementa insercao, busca, remocao e percurso em ordem. A remocao trata no sem filhos, com um filho e com dois filhos; nesse ultimo caso, substitui pelo sucessor em ordem e libera o no sucessor. O percurso em ordem apresenta IDs crescentes.

A fila insere no final e remove do inicio, mantendo ponteiros para inicio e fim. Ao remover o ultimo elemento, ambos voltam a nulo. A estrutura possui exibicao e liberacao.

entrada.c le linhas com fgets e converte inteiros com strtol, verificando erros e limites. IDs aceitos sao positivos e cabem em int; o menu aceita opcoes de 1 a 8. Linhas grandes demais, texto vazio e valores nao numericos sao rejeitados.

## 4. Testes e evidencias

O workflow do GitHub Actions executou com sucesso no commit 1b0f346. GCC compilou o aplicativo e os testes usando C11, -Wall, -Wextra e -pedantic.

A suite automatizada verificou insercao, busca, IDs duplicados e zero, ordem crescente, remocao de folha, no com um filho e no com dois filhos, IDs ausentes, comportamento FIFO, repeticao de IDs e fila vazia.

A integracao cadastrou os IDs 40, 20 e 60 pelo menu e confirmou listagem 20, 40, 60. Tambem rejeitou o ID textual abc, rejeitou texto com 80 caracteres e verificou insercao/remocao na fila e fila vazia.

O roteiro lista testes manuais adicionais, como texto vazio e valor acima de INT_MAX, que nao foram registrados como executados.

## 5. Analise de complexidade

Seja n o numero de registros na BST, h sua altura, q o numero de IDs na fila e L o tamanho do texto.

Operacao - Complexidade
Busca, insercao e remocao na BST - O(h)
Caso medio equilibrado - O(log n)
BST degenerada, pior caso - O(n)
Percurso em ordem - O(n)
Enfileirar e desenfileirar - O(1)
Exibir fila - O(q)
Validar ou copiar texto - O(L)
Memoria - O(n + q)

A BST nao e auto-balanceada. A ordem de insercao pode produzir uma arvore degenerada; nesse caso, operacoes chegam a O(n).

## 6. Conclusao e ferramentas utilizadas

A implementacao aplica ponteiros, alocacao dinamica, modularizacao e as estruturas solicitadas. A suite e o workflow confirmaram os casos principais da BST, da fila, da validacao de texto longo, da fila vazia e da listagem ordenada. A principal limitacao e a ausencia de balanceamento automatico.

Utilizei uma ferramenta de IA como apoio para organizar o repositorio e redigir/revisar partes do codigo, dos testes e deste relatorio. O uso foi registrado no repositorio. O codigo deve ser explicado durante a demonstracao.